

Typischer Ablauf: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

1) Joins

1.1 INNER JOIN – nur passende Tupel

Aufgabe: Produktname + Kategorienname

```
SELECT p.productname, c.categoryname
FROM products p JOIN categories c ON p.categoryid = c.categoryid;
```

1.2 LEFT JOIN – alles links, auch ohne Match

Aufgabe: Alle Kunden + ihre Bestellungen

```
SELECT c.customername, o.orderid
FROM customers c LEFT JOIN orders o ON c.customerid = o.customerid;
```

Ohne Beziehung (klassisch):

```
WHERE o.orderid IS NULL
```

2) Subqueries – häufigste Muster

2.1 Subquery als Tabelle (Aggregation)

Aufgabe: Produkte mit > 100 verkauften Einheiten

```
SELECT p.productname, x.totalsold
FROM (SELECT productid, SUM(quantity) totalsold
      FROM orderdetails GROUP BY productid) x
JOIN products p ON p.productid = x.productid
WHERE x.totalsold > 100;
```

2.2 Subquery in WHERE IN

Aufgabe: Kunden mit mindestens einer Bestellung

```
SELECT customername FROM customers
WHERE customerid IN (SELECT customerid FROM orders);
```

2.3 Korrelierte Subquery

Aufgabe: Produkte teurer als Durchschnitt

```
SELECT productname, price FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

3) Mengenoperationen

3.1 UNION – Vereinigung

Aufgabe: Länder von Kunden **oder** Lieferanten

```
SELECT country FROM customers UNION SELECT country FROM suppliers;
```

UNION ALL → mit Duplikaten

3.2 INTERSECT – Schnittmenge

Aufgabe: Länder mit Kunden **und** Lieferanten

```
SELECT country FROM customers INTERSECT SELECT country FROM suppliers;
```

3.3 EXCEPT – Differenz

Aufgabe: Länder mit Kunden, aber ohne Lieferanten

```
SELECT country FROM customers EXCEPT SELECT country FROM suppliers;
```

Von Text → DB - Durchgängiges Mini-Beispiel

1) Textbeschreibung

„Ein Kunde kann viele Bestellungen haben. Jede Bestellung gehört genau zu einem Kunden.“

2) Konzeptionelles Modell (ERM)

```
@startuml
class "Kunde" as kunde {
    Kunden-ID <<CK>>
    Name
}
class "Bestellung" as bestellung {
    Bestell-ID <<CK>>
    Bestelldatum
}
kunde "gehört zu \n 1"--"hat \n *" bestellung
@enduml
```

3) Logisches Modell (Relationen)

```
@startuml
class customers {
    customerid <<CK>>
    customername
}
class orders {
    orderid <<CK>>
    orderdate
    customerid
}
orders --> customers : "customerid: \n customerid"
@enduml
```

4) Physisches Modell (SQL DDL)

```
CREATE TABLE customers (
    customerid INTEGER PRIMARY KEY,
    customername VARCHAR(50) NOT NULL
);

CREATE TABLE orders (
    orderid INTEGER PRIMARY KEY,
    orderdate TIMESTAMP,
    customerid INTEGER NOT NULL,
    FOREIGN KEY (customerid)
        REFERENCES customers(customerid)
);
```

Mini-Merkliste

- ERM: Entitäten + Beziehung + Kardinalitäten
- Logisch: Tabellen + (F)K als Attribute + Pfeile über FK
- Physisch: PRIMARY KEY, FOREIGN KEY, NOT NULL, Constraints